EXP NO: 9 MODEL DEPLOYMENT: REST API WITH FLASK AND CONTAINERIZATION WITH DOCKER

Date: _____

AIM:

To train a Machine Learning classification model, deploy the trained model as a REST API using Flask, and package the application in a Docker container for portable and consistent execution.

ALGORITHM:

1. Import the required libraries and load the Iris dataset.
2. Train a Random Forest classifier using the Iris flower measurements and labels.
3. Save the trained model to a model.pkl file using Joblib.
4. Create a Flask web application and load the saved model.
5. Create a GET / endpoint to check whether the API is running.
6. Create a POST /predict endpoint that accepts four flower measurements in JSON format.
7. Validate the input, predict the flower class, and return a JSON response.
8. Write requirements.txt and a Dockerfile to package the application.
9. Build the Docker image, run a container, and test the API using a POST request.

PROJECT STRUCTURE:

```
model_deployment/
|-- train.py
|-- app.py
|-- requirements.txt
|-- Dockerfile
```

CODE:

1. train.py - Train and save the model

```
from sklearn.datasets import load_iris
from sklearn.ensemble import RandomForestClassifier
import joblib

iris = load_iris()
X, y = iris.data, iris.target

model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X, y)

joblib.dump(model, "model.pkl")
print("Model trained and saved successfully!")
```

2. app.py - Deploy the model with Flask

```
from flask import Flask, request, jsonify
import joblib

app = Flask(__name__)
model = joblib.load("model.pkl")
class_names = ["setosa", "versicolor", "virginica"]
```

```
@app.route("/", methods=["GET"])
def home():
    return jsonify({"message": "ML Model API is running"})

@app.route("/predict", methods=["POST"])
def predict():
    data = request.get_json(silent=True)
    if not isinstance(data, dict) or "features" not in data:
        return jsonify({"error": "Provide JSON with a features list"}), 400

    features = data["features"]
    if (not isinstance(features, list) or len(features) != 4
            or any(type(x) not in (int, float) for x in features)):
        return jsonify({"error": "features must have 4 numbers"}), 400

    result = int(model.predict([features])[0])
    return jsonify({"prediction": result,
                    "flower": class_names[result]})

if __name__ == "__main__":
    app.run(host="0.0.0.0", port=5000)
```

3. requirements.txt

```
flask>=3.0,<4
scikit-learn>=1.4,<2
joblib>=1.3,<2
```

4. Dockerfile

```
FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY train.py app.py ./
RUN python train.py
EXPOSE 5000
CMD ["python", "app.py"]
```

EXECUTION PROCEDURE:

Step 1: Open a terminal in the model_deployment folder.

Step 2: Install packages and train the model.

```
pip install -r requirements.txt
python train.py
```

Step 3: Start the API locally (optional local test).

```
python app.py
```

Step 4: Build the Docker image (Docker Desktop must be running).

```
docker build -t iris-model .
```

Step 5: Run the Docker container. Stop the local Flask server first if port 5000 is already being used.

```
docker run --rm -p 5000:5000 iris-model
```

Step 6: Send a POST request to the REST API in a new terminal.

Windows PowerShell:

```
Invoke-RestMethod -Uri http://localhost:5000/predict `
  -Method POST -ContentType "application/json" `
  -Body '{"features":[5.1,3.5,1.4,0.2]}'
```

Linux / macOS terminal:

```
curl -X POST http://localhost:5000/predict \
  -H "Content-Type: application/json" \
  -d '{"features":[5.1,3.5,1.4,0.2]}'
```

OUTPUT:

Training output:

```
Model trained and saved successfully!
```

GET http://localhost:5000/

```
{"message": "ML Model API is running"}
```

POST http://localhost:5000/predict (sample expected response)

```
{
    "flower": "setosa",
    "prediction": 0
}
```

Class labels: 0 = Setosa, 1 = Versicolor, 2 = Virginica.

RESULT:

The Iris classification model was trained and saved successfully. A REST API was created using Flask to accept input features and return predictions in JSON format. The application was containerized with Docker, allowing the model and its dependencies to run in a portable environment. The sample request predicts the Iris Setosa class.

Note: The output shown is an expected sample from the provided code; local and Docker execution should be performed to obtain actual screenshots for a laboratory record. The built-in Flask server is used here for demonstration, not production deployment.